

Airbnb: Delivery 1

Daniel Mateo Ballesteros Molina
20221020102

Santiago Gómez Reyes
20221020098

Esteban Alexander Bautista Solano
20221020089

Daniel David Cuellar Cangrejo
20221020081

Juan Andrés Jiménez Palomino
20221020087

September 2026

Contents

1	Introduction	3
2	System Description	4
2.1	Business Problem	4
2.2	Actors	4
2.2.1	Human users	4
2.2.2	External Services	6
2.3	System Diagram	7
2.4	Main Use Cases	8
2.4.1	Account Management	8
2.4.2	Accommodation Discovery	10
2.4.3	Reservation and Guest Experience	11
2.4.4	Host Management	15
2.4.5	Administration	17
2.4.6	Customer Support	18
2.5	System Boundaries	19
3	Functional Requirements	21
3.1	Rental Management	21

3.2	Accommodation Search	21
3.3	Security and Authentication	22
4	Non-Functional Requirements	23
5	Workload Model	25
5.1	Traffic and load	25
5.2	User Activity	26
5.3	Read and Write Ratio	26
5.4	Performance and reliability	27
5.5	Data	28
5.6	Media	28
5.7	Search	28
5.8	Geography	29

1 Introduction

In the present document, it will be boarded the system description of Airbnb, which will the appropriate analysis of its business problem, which actors take part of it, describe its main Use Cases, and define the boundaries within the system, i.e., what are the limits of the designed and development of the system.

Then, it will be redacted and described the Functional and Non-Functional Requirements that the system accomplishes in a real-life situation. The values of the Non-Functional Requirements, will be defined using Airbnb Workload Model, which will appear in its own section in the document.

The boundaries and limits, will also specify how will the real Non-Functional Requirements be taken into account for its simulation.

2 System Description

In the present section, there will be a description of Airbnb system, which will take further the business problem it was designed to solve, as well as its main actors, and how they interact within the system described in the corresponding Use Cases.

Finally, there will be defined the System Boundaries, i.e, what is inside of the scope of our system and what is explicitly outside of it.

2.1 Business Problem

The problem that Airbnb solves is referred to the fragmented, inefficient and uncertain access to short-term accomodations. It gives people who are in need of a place to stay, and people with those accomodations or properties available. The hosts might find problems finding or reaching potential guests, managing bookings and monetizing after all the stay, and guests might struggle finding said place.

Airbnb provides a platform in which guests are benefited with a large range and access of accomodation options, and hosts can get additional income and a wider spectre of possible guests around the world.

Its operational challenge at scale, comes when one must maintain availability accuracy (concurrency), transaction integrity, trust, and service reliability accross a global two-sided marketplace. Also, there might be some problems taking compliance into account.

2.2 Actors

Taking actors, as the human users and the external services that are interacting with our application, the following schemas describe their role on it, how much the interact with the application, and what could they expect.

2.2.1 Human users

User	Role	Interaction Frequency	Expectations Regarding the System
Visitors	They browse listing and explore accomodations without having an account.	Their interaction frequency is high, as they would use the main requirements of the application without counting logged in users.	They would expect fast searches, useful filters, accurate prices, clear property descriptions, the map-base discovery and a simple registration process.
Guests	They search for accomodation, book their stays, pay, communicate with hosts and manage their reservations.	Their interaction is high during the accomodation discovery and booking. After their stay, it would be more ocassional.	They might expect accurate availability, transparent pricings, reliable confirmations, clear cancellation and effective support.
Hosts	They publish and manage their property listings, set prices and availability, accept or manage reservations, communicate with the guests and receive payouts.	Their interaction frequency is frequent, daily-like, it would depend a lot on the reservations, but fewer than visitors and guests.	They would expect an easy listing management, synchronized calendars (to reduce concurrence), pricing controls, timely notifications, secure payouts and booking management.
Auditors	They review transactions, access records, verify compliance and examine audit trails.	Their interaction frequency is periodic, with more frequency when there is a programmed audit or investigation.	They would expect accurate and traceable records, controlled access to sensitive data, reliable reports and evidence of relevant changes and transactions.

User	Role	Interaction Frequency	Expectations Regarding the System
Administrators	They configure platform settings, manage accounts and permissions, oversee listings, and supervise operational processes.	The interaction frequency is frequent during operational work, it would be expected to be daily.	They would expect to have role-based access control, monitoring tools, reliable reporting, secure configuration and traceability of administrative actions.
Customer Support	They assist guests and hosts with booking issues, refunds, complaints and account access.	Their interaction frequency is high during working shifts, peaking at incidents or high demand on the application.	They would expect fast access to reservation history, communication records, relevant policies and clear escalation procedures.

Table 1: Human users roles, interaction frequency and system expectations

2.2.2 External Services

User	Role	Interaction Frequency	Expectations Regarding the System
Geolocation / Maps API	It's in charge of converting addresses and coordinates, displaying properties on maps and supporting location-based searches.	Its interaction would be frequent during map browsing and location searches.	The expectations would be to get accurate coordinates, responsive queries, consistent location data and high availability.

User	Role	Interaction Frequency	Expectations Regarding the System
Payment Gateway	It's in charge of processing guests payments and refunds.	Its interaction would be event-driven like, being invoked during the checkout, refunds and other transactions, making it less active than other services.	The expectations rely on having a secure processing, reliable transactions and clear handling of failed and pending transactions.
Authentication Service	It's in charge of authenticating users, managing sessions and tokens.	The interaction frequency would be higher during login and session renewal, and less when sessions remain active.	The expectations would be to secure credential handling, have a reliable access control, low latency and protection against unauthorized access.
Notification Service	It's in charge of delivering booking confirmations, reservation changes, reminders, messages and payment alerts.	Its interaction would also be event-driven like, peaking during booking peaks.	The expectations would be to deliver them on time, have a tracking on delivery status, preventing duplicates and channel specific-failure handling.

Table 2: External users roles, interaction frequency and system expectations

2.3 System Diagram

Here it is a description of the system into a diagram, which shows how the actors interact within the application in a graphic way. It gives a workflow with inputs and outputs and clear interaction between the system components.

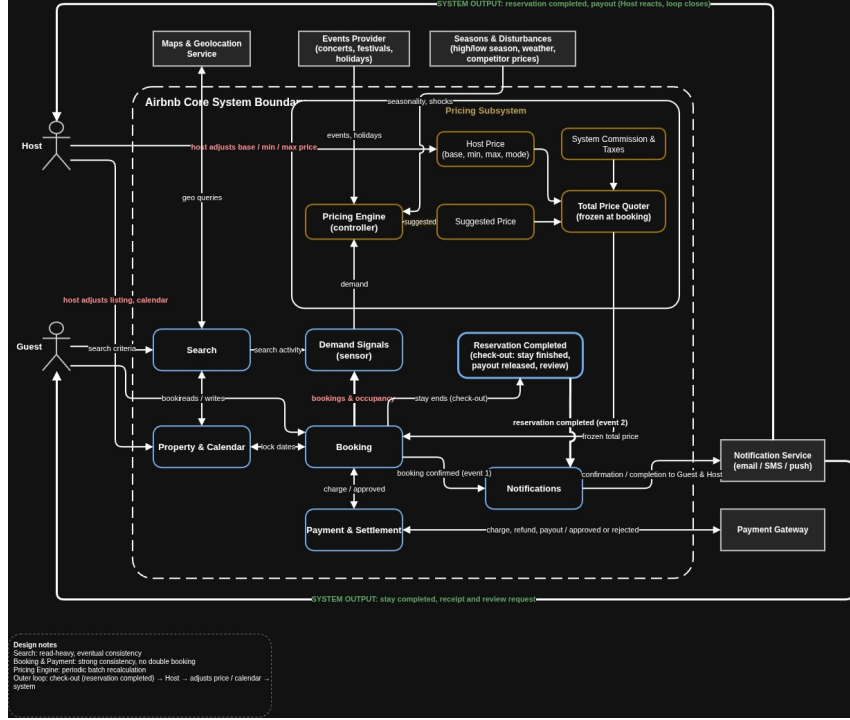


Figure 1: System Diagram.

2.4 Main Use Cases

The following use cases were the most representative interactions of the system defined, focusing on the main business processes of the platform rather than technical operations performed by external services, i.e., supporting services such as the Payment Gateway, Maps Service, Authentication Service, and Notification Service participate in these use cases when required.

2.4.1 Account Management

Register

- **Actors:** Guest, Host, Authentication Service.
- **Preconditions:**
 - The user does not already have an account with the provided credentials.
 - The registration service is available.
- **Main Flow:**
 1. The user selects the registration option.
 2. The system requests the required registration information.

3. The user provides the requested information.
4. The system validates the information.
5. The Authentication Service creates and validates the user credentials.
6. The system creates the user account.
7. The system confirms that the registration was successful.

- **Alternative Flows:**

- The provided information is invalid or incomplete.
 - The provided email or credentials are already associated with an existing account.
 - The Authentication Service is unavailable.

- **Postconditions:**

- The user's credentials are registered with the authentication mechanism.

Log In

- **Actors:** Guest, Host, Administrator, Customer Support, Authentication Service.

- **Preconditions:**

- The user has an existing account.
 - The authentication service is available.

- **Main Flow:**

1. The user enters their credentials.
2. The system sends the credentials to the Authentication Service.
3. The Authentication Service validates the credentials.
4. The system creates an authenticated session.
5. The system grants access according to the user's role.

- **Alternative Flows:**

- The credentials are invalid.
 - The Authentication Service is unavailable.

- **Postconditions:**

- The user is authenticated.

2.4.2 Accommodation Discovery

Search Accommodation

- **Actors:** Guest, Maps Service.
- **Preconditions:**
 - The accommodation search service is available.
- **Main Flow:**
 1. The guest enters a destination or location.
 2. The guest specifies optional dates, number of guests, and search criteria.
 3. The system queries available accommodations.
 4. The system applies the selected filters and criteria.
 5. The Maps Service provides geographical information when required.
 6. The system displays the matching accommodations.
- **Alternative Flows:**
 - No accommodations match the search criteria.
 - The requested location cannot be resolved.
- **Postconditions:**
 - The guest receives a list of accommodations matching the search criteria.

View Accommodation Details

- **Actors:** Guest, Maps Service.
- **Preconditions:**
 - The accommodation exists and is available for viewing.
- **Main Flow:**
 1. The guest selects an accommodation from the search results.
 2. The system retrieves pricing, availability, amenities, and ratings.
 3. The Maps Service provides the location information when required.
- **Alternative Flows:**

- The accommodation is no longer available.
- The requested accommodation information cannot be retrieved.

- **Postconditions:**

- The guest can review the accommodation information.

2.4.3 Reservation and Guest Experience

Book Accommodation

- **Actors:** Guest, Host, Authentication Service, Payment Gateway, Notification Service.

- **Preconditions:**

- The guest is authenticated.
- The accommodation is active.
- The selected dates are available.

- **Main Flow:**

1. The guest selects an accommodation and dates.
2. The system verifies availability.
3. The system calculates the reservation price.
4. The guest confirms the reservation.
5. The system requests payment processing.
6. The Payment Gateway processes the transaction.
7. The system creates the reservation.
8. The system updates the accommodation availability.
9. The Notification Service sends the reservation confirmation.

- **Alternative Flows:**

- The accommodation becomes unavailable before confirmation.
- The payment is rejected.
- The payment remains pending.
- The reservation cannot be created.

- **Postconditions:**

- The reservation is registered with its corresponding status.
- The accommodation availability is updated.
- The guest and host receive the corresponding notification.

Cancel Reservation

- **Actors:** Guest, Host, Payment Gateway, Notification Service.

- **Preconditions:**

- The reservation exists.
- The reservation can still be cancelled according to its cancellation policy.

- **Main Flow:**

1. The user selects the reservation to cancel.
2. The system verifies the cancellation conditions.
3. The system changes the reservation status.
4. The system updates the accommodation availability.
5. The system determines whether a refund is applicable.
6. The Payment Gateway processes the refund when applicable.
7. The Notification Service sends the cancellation notification.

- **Alternative Flows:**

- The reservation cannot be cancelled under its current conditions.
- The refund cannot be processed.
- The cancellation notification cannot be delivered.

- **Postconditions:**

- The reservation is marked as cancelled.
- The accommodation availability is updated.
- The applicable refund is registered.

Communicate with Host or Guest

- **Actors:** Guest, Host, Notification Service.
- **Preconditions:**
 - The user is authenticated.
- **Main Flow:**
 1. The user selects the conversation.
 2. The user writes and sends a message.
 3. The system stores the message.
 4. The Notification Service notifies the receiver.
 5. The receptor receives the message.
- **Alternative Flows:**
 - The receptor cannot be contacted.
 - The message cannot be delivered.
- **Postconditions:**
 - The message is stored in the corresponding conversation.
 - The receptor is notified when possible.

Rate Accommodation

- **Actor:** Guest.
- **Preconditions:**
 - The guest has completed an eligible reservation.
 - The guest has not already submitted a rating for the reservation.
- **Main Flow:**
 1. The guest selects the completed reservation.
 2. The system displays the rating form.
 3. The guest provides a rating and optional comments.
 4. The system validates and stores the rating.

5. The system updates the accommodation's rating information.

- **Alternative Flows:**

- The reservation is not eligible for rating.
- The guest has already submitted a rating.

- **Postconditions:**

- The accommodation's rating information is updated.

Report a Problem

- **Actors:** Guest, Host, Customer Support.

- **Preconditions:**

- The user is authenticated.
- The reported issue can be associated with a reservation, accommodation, or account.

- **Main Flow:**

1. The user selects the report option.
2. The user provides the relevant information.
3. The system creates a support case.
4. Customer Support receives the case.
5. The system notifies the user of the case status.

- **Alternative Flows:**

- The provided information is incomplete.
- The reported issue cannot be associated with the provided information.

- **Postconditions:**

- A support case is created processed by Customer Support.

2.4.4 Host Management

Publish Accommodation

- **Actors:** Host, Maps Service.
- **Preconditions:**
 - The host is authenticated.
 - The host has the required information to create a listing.
- **Main Flow:**
 1. The host selects the option to publish an accommodation.
 2. The host enters the accommodation information.
 3. The Maps Service validates or resolves the location.
 4. The system validates the information.
 5. The system creates the accommodation listing.
- **Alternative Flows:**
 - Required information is missing.
 - The location cannot be resolved.
- **Postconditions:**
 - The accommodation listing is stored.

Manage Accommodation Listing

- **Actors:** Host, Maps Service.
- **Preconditions:**
 - The host is authenticated.
 - The accommodation belongs to the host.
- **Main Flow:**
 1. The host selects an accommodation listing.
 2. The host modifies its information, pricing, or availability.
 3. The system validates the changes.

4. The Maps Service updates geographical information when required.
5. The system stores the changes.
6. The system updates the listing information.

- **Alternative Flows:**

- The provided information is invalid.
 - The requested dates conflict with existing reservations.
 - The location information cannot be validated.

- **Postconditions:**

- The accommodation listing contains the updated information.
 - Existing reservations remain protected from incompatible changes.

Manage Reservations

- **Actor:** Host.

- **Preconditions:**

- The host is authenticated.
 - The host has one or more reservations associated with their accommodations.

- **Main Flow:**

1. The host accesses their reservations.
2. The system displays the reservations and their current status.
3. The host selects a reservation.
4. The host performs an available management action.
5. The system validates the action.
6. The system updates the reservation status when applicable.
7. The Notification Service informs the affected users.

- **Alternative Flows:**

- The reservation status does not allow the requested action.

- **Postconditions:**

- The reservation reflects the performed action.
 - The affected users are notified when applicable.

2.4.5 Administration

Manage Users

- **Actors:** Administrator.
- **Preconditions:**
 - The administrator is authenticated and authorized.
- **Main Flow:**
 1. The administrator searches for a user.
 2. The system displays the user's information and status.
 3. The administrator selects an administrative action.
 4. The system applies the requested action.
- **Alternative Flows:**
 - The administrator does not have sufficient permissions.
 - The user cannot be found.
- **Postconditions:**
 - The action is recorded.

Manage Accommodations

- **Actors:** Administrator.
- **Preconditions:**
 - The administrator is authenticated and authorized.
- **Main Flow:**
 1. The administrator searches for an accommodation.
 2. The system displays the accommodation information and status.
 3. The administrator selects an administrative action.
 4. The system validates the administrator's permissions.
 5. The system applies the requested action.
 6. The system records the administrative action.

- **Alternative Flows:**

- The accommodation cannot be found.
- The administrator does not have sufficient permissions.
- The requested action is not permitted.

- **Postconditions:**

- The accommodation reflects the administrative action.
- The action is recorded for traceability.

2.4.6 Customer Support

Manage Support Cases

- **Actors:** Customer Support, Guest, Host.

- **Preconditions:**

- A support case exists.
- Customer Support is authenticated and authorized.

- **Main Flow:**

1. Customer Support accesses the case.
2. The system displays the related user, reservation, and case information.
3. Customer Support investigates and communicates the reported issue with the affected user.
4. The case status is updated.

- **Alternative Flows:**

- Additional information is required from the user.

- **Postconditions:**

- The support case has an updated status.
- The actions taken by Customer Support are recorded.

Handle Refunds

- **Actors:** Customer Support, Guest, Payment Gateway, Notification Service.
- **Preconditions:**
 - A reservation or transaction eligible for a refund exists.
 - Customer Support is authenticated and authorized.
- **Main Flow:**
 1. Customer Support accesses the relevant case or reservation.
 2. The system determines the applicable refund conditions.
 3. Customer Support confirms the refund.
 4. The system sends the refund request to the Payment Gateway.
 5. The Payment Gateway processes the refund.
 6. The system records the refund status.
 7. The user is notified of the result.
- **Alternative Flows:**
 - The transaction is not eligible for a refund.
 - The refund remains pending.
- **Postconditions:**
 - The refund is recorded with its corresponding status.
 - The affected user is notified.

2.5 System Boundaries

The system boundary is defined around the core rental marketplace, including the capabilities required to publish, discover, and manage rental agreements between hosts and guests. The system includes rental information such as pricing, dynamic pricing rules, dates, check-in and check-out agreements, terms of use, and specific guest conditions. It also includes the host interface for publishing and managing properties, including percentage-based charges, and the guest interface for searching and reviewing available rentals. The system must also provide authentication and security, as well as the raw rental listing required to support the marketplace. Map-based catalog visualization, system filters, and favorites are considered within

the boundary as secondary capabilities that support the accommodation discovery process.

The boundary also includes capabilities classified as “Could Have” where they involve the platform’s own operational functionality, particularly administration, monitoring, and auditing, including the visualization of external transactions. Customer support is likewise considered within the system boundary, although it is not part of the fundamental rental workflow. The payment gateway, however, is treated as an external system: payment integration may be included as a future capability, but the actual financial transaction processing is performed outside the system. Similarly, external authentication or other third-party services may interact with the platform without becoming part of its internal scope.

The system explicitly excludes cleaning, maintenance, and other third-party property services, which were classified as “Won’t Have”. These activities may be relevant to the real-world rental process, but they are not managed by the application itself. More generally, the system boundary ends at the point where an external organization or service performs an operational activity that the application does not manage. The details of each included capability, including its functional behavior and requirements, will be specified later in the functional requirements section.

3 Functional Requirements

In this section, there are presented 10 functional requirements describing what the system must do. They are focused to fulfill all of the required characteristics.

3.1 Rental Management

Here are described some functional requirements related to rental management.

ID	Functional Requirement
FR01	The system shall allow hosts to publish rental properties by providing the required property information, pricing, availability dates, and usage conditions.
FR02	The system shall allow hosts to manage their published rental properties by modifying their property information, pricing, availability, and usage conditions while preserving existing reservations.
FR03	The system shall calculate the rental price for a reservation according to the configured pricing rules and applicable dynamic rates for the selected dates.
FR04	The system shall allow guests to select rental dates and the number of guests before creating a reservation, subject to the property's configured availability and capacity.
FR05	The system shall record check-in and check-out agreements for each confirmed reservation before the reservation is completed.
FR06	The system shall allow hosts to define specific conditions for guests, including guest capacity and additional conditions associated with extra charges, before publishing a rental property.

Table 3: Rental Management Requirements

3.2 Accommodation Search

Here are described the functional requirements related to accommodation search.

ID	Functional Requirement
FR07	The system shall allow guests to search for rental properties using destination, rental dates, and number of guests as search criteria.
FR08	The system shall display the raw rental listing information for each available property returned by a search, including its location, price, availability, and configured conditions.

Table 4: Accommodation Search Requirements

3.3 Security and Authentication

Here are described the funtional requirements related to security and authentication.

ID	Functional Requirement
FR09	The system shall authenticate registered users by validating their credentials before granting access to protected system functionality.
FR10	The system shall restrict access to protected functionality according to the authenticated user's assigned role.

Table 5: Security ad Authentication Requirements

4 Non-Functional Requirements

In this section, there will be defined the Non-Functional Requirements that the system must achieve and support. The metrics where used from the following description of Airbnb’s Workload Model. For its simulation, values will vary, as it is just a “shorter” version of the application, which will take into account its basic functionalities.

ID	Requirement
NFR-01	The system shall support more than 220 countries and regions, with approximately 23,000 concurrent active users under normal operating conditions and approximately 60,000 concurrent active users during peak conditions.
NFR-02	The system shall support an average workload of approximately 934,000 requests per hour under normal operating conditions.
NFR-03	The system shall support a peak workload of approximately 9.3 million requests per hour during peak operating periods.
NFR-04	The system shall sustain peak workloads for periods of approximately 5 minutes without degradation below the defined performance and availability targets.
NFR-05	The system shall process at least 95% of user-facing requests within approximately 1.5 seconds at the 95th percentile under normal operating conditions.
NFR-06	The system shall maintain an availability of at least 99.9%, corresponding to no more than approximately 43.2 minutes of unavailability per calendar month.
NFR-07	The system shall support an initial data volume of approximately 1 GB and accommodate an estimated annual data growth of approximately 8–10%.
NFR-08	The system shall accommodate annual data growth of approximately 8–10% of the preceding year’s stored data volume without requiring architectural changes to support the projected workload.

ID	Requirement
NFR-09	The system shall serve users across more than 220 countries and regions and shall distribute its infrastructure and traffic-handling mechanisms geographically to minimize latency for users in North America, EMEA, Latin America, and Asia Pacific.

Table 6: Non-Functional Requirements

The Airbnb values in the previous table represent the scale of the real platform, so they would be unnecessarily large for a small academic simulation. For this project, the workload will be scaled down to 100 registered users, with approximately 10 concurrent active users under normal conditions and 25 during peak conditions. The simulation will generate approximately 500 requests per hour during normal operation and up to 1,200 requests per hour during peak periods, with peak periods modeled as lasting approximately 5 minutes. The response-time target will be 95th percentile below 2 seconds, while the system will target 99% availability, which is more appropriate for a small local or academic deployment than Airbnb’s production-scale availability target.

For data and geographic scope, the simulation will initially contain approximately 100 MB of application data, with an expected growth of approximately 20% per year as new users, listings, reservations, and related records are added. The system will be designed to serve a single geographic region, rather than reproducing Airbnb’s worldwide distribution, and therefore no geographically distributed infrastructure will be required. These values provide a manageable workload for the project while preserving the same types of constraints represented by the original Airbnb workload model: user concurrency, request volume, peak behavior, response time, availability, data growth, and geographic scope.

5 Workload Model

In the present section, the workload model of Airbnb will be described, giving the different parameters with its corresponding value. Each value is justified with its proper reference, the data that wasn't publicly available was estimated taking into account other parameters and data, its justification will be found under the corresponding table.

5.1 Traffic and load

The following table shows the value of the traffic that Airbnb experiments.

Parameter	Value
Average requests/hour	$\approx 934,000$ page requests/hour [1]
Peak requests/hour	≈ 9.3 million page requests/hour [1, 2]
Peak duration	≈ 5 minutes [2]
Burst rate	$10\times$ normal; $20\times$ stress case [2]
Seasonal traffic multiplier	$\approx 1.1\times$ [3]

The value of average requests/hour was estimated using Semrush, which reports 93.03M monthly visits and 7.23 pages/visit. $93.03M \times 7.23/720 \approx 934,000$ page requests/hour. This represents observable web page traffic, not Airbnb's complete internal service-to-service traffic [1].

The peak requests/hour were estimated. Taking into account that the average web workload above is multiplied by a conservative $10\times$ burst factor. Airbnb engineering has documented traffic spikes of approximately $10\text{--}20\times$ during severe events. $934,000 \times 10 \approx 9.34$ million requests/hour [1, 2].

The burst rate was derived from Airbnb engineering, which has documented QPS increases in the $10\text{--}20\times$ range. $10\times$ is used as the normal capacity-planning burst and $20\times$ as an extreme stress scenario. Source: Airbnb Engineering [2].

The seasonal traffic multiplier was estimated. Airbnb states that its business is seasonal and that Nights and Seats Booked are generally highest in Q1–Q3, with Q3 being the peak travel season in North America and EMEA. A $1.1\times$ multiplier provides a conservative approximation of normal seasonal variation without treating the annual peak as an outage-level burst [3].

5.2 User Activity

The following table shows the value of the user activity in the application.

Parameter	Value
Average session duration	10 min 42 s [1]
Requests per session	7.23 page requests/session [1]
Concurrent users (average)	$\approx 23,000$ web sessions [1]
Concurrent users (peak)	$\approx 60,000$ web sessions [1]

The average session duration was recorded from a third-party measurement. Semrush reports an average visit duration of 10:42 for airbnb.com in August 2026 [1].

The amount of requests per session was used from a third-party measurement. Semrush reports 7.23 pages per visit. This is interpreted as user-facing page requests rather than internal microservice requests [1].

The concurrent users were estimated, using approximately 93.03M monthly visits and an average session duration of 10.7 minutes:

$$\frac{93.03M}{30.4 \text{ days}} \times \frac{10.7 \text{ min}}{1440 \text{ min/day}} \approx 22,800$$

This estimates simultaneously active web sessions and excludes users interacting exclusively through native applications. Source: Semrush traffic and session-duration data [1].

The peak of the concurrent users was estimated. A peak concurrency factor of approximately $2.5\times$ the average is used: $22,800 \times 2.5 \approx 57,000$, rounded to approximately 60,000. The factor is intentionally lower than the $10\times$ request burst because more requests per user can occur without a proportional increase in active users [1].

5.3 Read and Write Ratio

The following table shows the value of the user read and write ratio of the application.

Parameter	Value
Read/write ratio	$\approx 95 : 5$ [4]
Request distribution by endpoint	Search/browse 60%; listing/details 20%; account/auth 8%; booking/payment 5%; messaging 4%; other 3% [5]
Cache hit ratio	$\approx 85\%$ of cacheable reads [5]

The read and write ratio was estimated from Airbnb internal infrastructure. Airbnb’s Mussel key-value store reports average read QPS above 800k and write QPS above 35k, corresponding to approximately 96:4. A rounded 95:5 ratio is used for the general workload model because search and discovery are substantially more read-intensive than booking and data modification [4].

The request distribution was estimated. The distribution emphasizes search because Airbnb system-design and engineering material identify search/discovery as the dominant read-heavy workload, while booking/payment represents a smaller but consistency-critical workload. Percentages are architectural assumptions for capacity modeling [5].

The cache hit ratio was estimated. Airbnb publicly documents extensive caching and derived-data infrastructure, but does not publish a single platform-wide cache hit ratio. An 85% hit ratio is a reasonable architectural assumption for frequently accessed listing/search data and should be treated as a tunable model parameter [5].

5.4 Performance and reliability

The following table shows the value of the performance of the application.

Parameter	Value
Availability	$\geq 99.9\%$ [4]
Response time (p95)	≈ 1.5 s user-facing target [4, 5]

The availability was used from a report for a major Airbnb infrastructure component. Airbnb’s Mussel service reports greater than 99.9% availability. This value is used as a conservative platform workload target, but cannot be described as an officially published end-to-end Airbnb SLA [4].

The response time, within a 95 percentile was estimated. Even though Airbnb does not publish a global end-to-end p95. Mussel reports average p95 read latency below 8 ms, demonstrating backend-storage performance, while an end-user request

traverses multiple services, networks, caches and rendering layers. Therefore, approximately 1.5 s is used as a user-facing workload target rather than incorrectly treating 8 ms as total request latency [4, 5].

5.5 Data

The following table shows the value of the data growth in the application.

Parameter	Value
Data growth	$\approx$ 8–10%/year [3]

The data growth was estimated from reported business growth. Airbnb reported 533M Nights and Seats Booked in 2025 versus 492M in 2024, an 8% increase. A baseline of 8–10% annual growth is therefore used for workload/data planning, recognizing that storage growth will not necessarily equal booking growth [3].

5.6 Media

The following table shows the value of the information related to the media files in the application.

Parameter	Value
Maximum payload size	10 MB/photo; 100 MB/video [6, 7]
File upload frequency	$\approx$ 9,000 photos/hour [8, 6]

The file upload frequency was estimated. Considering Airbnb has more than 9M active listings according to current third-party industry data. Assuming roughly one photo update per listing per year and several photos per update produces an order of magnitude of several thousand photo uploads/hour; 9,000/hour is used as a conservative media-workload estimate [8, 6].

5.7 Search

The following table shows the value of the searches in the application, and their complexity.

Parameter	Value
Search complexity	High; geo-search + dates + availability + price + filters + ranking/personalization [5]

The search complexity was oriented as qualitative analysis, due to the many procedures it requires. Airbnb search must combine geographic location, dates, availability, pricing, amenities and ranking/personalization. This is therefore a multi-dimensional read-heavy workload requiring indexes, caching and precomputed/derived data [5].

5.8 Geography

The following table shows the value of the geography scope in the application.

Parameter	Value
Geographic scope	More than 220 countries and regions [3]
Traffic by region	North America 30%; EMEA 40%; Latin America 17%; Asia Pacific 13% [3]

The traffic by region was used from a reported business-workload proxy. Airbnb reports 2025 Nights and Seats Booked by host-listing region: North America 158M (30%), EMEA 215M (40%), Latin America 90M (17%), and Asia Pacific 70M (13%). These figures describe booking activity rather than HTTP traffic, so they should be identified as a traffic-distribution proxy [3].

References

- [1] Semrush. *Airbnb.com Website Traffic Statistics*. Traffic, pages per visit, and average visit duration data. 2026. URL: https://www.semrush.com/website/airbnb.com/overview/?source=trending-websites&utm_source=chatgpt.com (visited on 10/04/2026).
- [2] Airbnb Engineering. *Building Services at Airbnb, Part 3*. Traffic and QPS spike behavior. 2017. URL: <https://medium.com/airbnb-engineering/building-services-at-airbnb-part-3-ac6d4972fc2d> (visited on 10/04/2026).
- [3] Airbnb, Inc. *Annual Report 2025 (Form 10-K)*. Geographic scope, regional booking distribution, seasonality, and business growth. 2026. URL: <https://www.sec.gov/Archives/edgar/data/1559720/000155972026000004/abnb-20251231.htm> (visited on 10/04/2026).
- [4] Airbnb Engineering. *Mussel: Airbnb's Key-Value Store for Derived Data*. Read/write QPS, availability, latency, and derived-data infrastructure. 2020. URL: <https://medium.com/airbnb-engineering/mussel-airbnbs-key-value-store-for-derived-data-406b9fa1b296> (visited on 10/04/2026).
- [5] Educative. *Airbnb System Design*. Search architecture, workload characteristics, caching, and system-design considerations. 2024. URL: <https://www.educative.io/blog/airbnb-system-design> (visited on 10/04/2026).
- [6] Airbnb. *How to Take Great Photos for Your Listing*. Listing photo and media guidance. 2026. URL: <https://www.airbnb.com/resources/hosting-homes/a/how-to-take-great-photos-for-your-listing-687> (visited on 10/04/2026).
- [7] Airbnb. *Listing Verification Video Requirements*. Video upload requirements and maximum video size. 2026. URL: <https://www.airbnb.com/help/article/3776> (visited on 10/04/2026).
- [8] Search Logistics. *Airbnb Statistics*. Third-party Airbnb marketplace and listing statistics. 2026. URL: <https://www.searchlogistics.com/learn/statistics/airbnb-statistics/> (visited on 10/04/2026).